CompTIA SecAI+

Lab 1.4 — Data Integrity Issues

Using GPT OSS 20B in Open WebUI

Lab Setup

Tools that process structured data expect it in a specific format. When the input matches, the tool works; when it doesn't, results range from obvious errors to confusing failures. In this lab you will explore two classes of data integrity issue: loud issues that fail immediately and visibly, and silent issues that still produce valid data but change its meaning. You will use hashing and diff to detect both.

Download the files from Labfiles 1.4 (alert_file1.json, alert_file1 (copy).json, and alert_file2.json) and save them in home/kali/Labfiles1.4.

Before you begin, open Chat Controls (the sliders icon at the top-right of the chat), select Advanced Parameters, and apply these settings:

Parameter	Value
Max Tokens	2048
Every other parameter	Default (do not change)

Note: Do not change num_thread, num_batch, top_k, min_p, or any other advanced parameter — only Max Tokens should be changed. The others are not supported and cause a “property … is unsupported” error.

Note: The free tier allows about 8,000 tokens per minute, and a chat resends its whole history on every message. Start a New Chat for each exercise, and if you see a “request too large / tokens per minute” message, start a New Chat (or wait a minute) and continue.

Note: The model for this lab is GPT OSS 20B (Direct tab in the model dropdown).

1.4.1 Loud Data Integrity Issues

Loud data integrity issues cause an immediate, visible failure. You will run the same alert analysis against two JSON files: one that matches the expected format and processes successfully, and one with an unexpected change that breaks the format and causes the tool to fail outright.

Step 1: From the desktop, open Firefox and sign into Open WebUI.

Step 2: Download the files for this lab from Labfiles 1.4 and save them to home/kali/Labfiles1.4.

Step 3: From Open WebUI, start a new chat with **GPT OSS 20B**.

Step 4: Upload **alert_file1.json** from /home/kali/Labfiles1.4 directly into the chat, then enter the following prompt and note the response:

Prompt 1:

```
Analyze the uploaded alert file and provide a structured summary.
```

Insights: This alert file follows the expected JSON structure and formatting rules, so the model can parse it successfully and produce a structured summary without errors.

Step 5: Start a new chat, upload **alert_file1 (copy).json** from /home/kali/Labfiles1.4 into the chat, then enter the following prompt and note the response:

Prompt 2:

```
Analyze the uploaded alert file and provide a structured summary.
```

Insights: This alert fails to load because the JSON is no longer valid. A syntax error prevents the parser from reading the file, causing the tool to stop immediately. This is a loud integrity failure — the problem is obvious and blocks analysis entirely.

Step 6: From the left menu, open the **terminal**.

Step 7: In the terminal, run the following commands:

```
cd /home/kali/Labfiles1.4
sha256sum alert_file1.json "alert_file1 (copy).json"
```

Insights: Even though one alert file is a copy of the other, a different hash confirms something inside the file has changed. Since these files are generated by a tool, their contents should not be modified manually. Hashing doesn't tell you what changed, but it reliably confirms that a change occurred.

Step 8: From the desktop, open the Labfiles1.4 folder and open **alert_file1 (copy).json**.

Insights: The tlp:white value is supposed to be a string, but it is highlighted by the editor because it is not enclosed in double quotes. In JSON, string values must always be quoted, and without them the file no longer conforms to the format.

Step 9: In the file, enclose **tlp:white** in double quotes.

Note: The file may hang briefly when adding each quote.

Step 10: Save and close the file.

Step 11: In the terminal, run the hash command again:

```
cd /home/kali/Labfiles1.4
sha256sum alert_file1.json "alert_file1 (copy).json"
```

Insights: The hash values now match, confirming the alert file has been fully restored to its original contents.

You have successfully explored loud data integrity issues.

1.4.2 Silent Data Integrity Issues

Not all data integrity issues cause obvious failures. Data can be modified in ways that still produce valid JSON and let tools run normally, even though the meaning of the data has changed. These cases are dangerous because nothing breaks and there are no immediate signs anything is wrong. You will use hashing to detect that a file has changed, then inspect it to find what was altered.

Step 1: Switch to Open WebUI and start a new chat with **GPT OSS 20B**.

Step 2: Upload **alert_file2.json** from /home/kali/Labfiles1.4 directly into the chat, then enter the following prompt and note the response:

Prompt 1:

```
Analyze the uploaded alert file and provide a structured summary.
```

Insights: This alert processes successfully because the JSON is still valid, but the silent changes significantly affect how the model interprets the data. Lowering the threat level ID to 0 makes the model downplay the risk, while changing the filename influences how the activity is framed. Because nothing fails, these changes can easily go unnoticed.

Step 3: From the desktop, open the Labfiles1.4 folder and open **alert_file2.json**.

Insights: There are no obvious formatting errors or visual indicators that something is wrong. The JSON is valid and structured as expected, even though important values have been changed. This is what makes silent issues harder to detect — the data appears legitimate unless you already know what to look for.

Step 4: In the terminal, run the following commands:

```
cd /home/kali/Labfiles1.4
sha256sum alert_file1.json alert_file2.json
```

Insights: Comparing file hashes is especially effective for silent issues. Since alert_file2.json is a copy of alert_file1.json, the hash mismatch quickly confirms something changed.

Step 5: In the terminal, run the following commands:

```
python3 -m json.tool alert_file1.json > alert_file1.pretty.json
python3 -m json.tool alert_file2.json > alert_file2.pretty.json
```

Insights: JSON files are often stored on a single line, which makes it hard for diff to highlight individual changes. Pretty-printing expands the data into a line-by-line structure so diff can compare specific fields and values.

Step 6: Run the diff tool to compare the pretty-printed files from the previous step:

```
diff -u alert_file1.pretty.json alert_file2.pretty.json
```

Insights: The diff command compares two files line by line and highlights exactly what changed. Lines prefixed with "-" show values from the original file; lines prefixed with "+" show values from the modified version. This makes silent changes visible and easy to analyse.

Step 7: Restore the file from a known-good backup.

Insights: After identifying silent changes with hashing and diff, the safest response is to restore from a known-good backup rather than editing by hand. Because these changes don't break tools, there is no reliable point at which you can say the file is "correct" again, and manual edits risk introducing new changes. Restoring from backup is faster and more reliable, and keeps downstream tools and AI models trustworthy.

You have successfully explored silent data integrity issues.

Summary — Loud vs Silent

This lab demonstrated two classes of data integrity issue and how they affect tools and AI-assisted analysis.

	Loud Data Integrity Issues	Silent Data Integrity Issues
Indicators	Tools fail immediately with errors, making the issue obvious.	Tools and AI models process the data normally, trusting the modified values.
Why it matters	The failure provides a clear signal that something is wrong.	There are no errors or warnings, and AI systems treat poisoned data as ground truth.
Best defences	Strict format validation; schema checks; automated parsing and early rejection; restore from a known-good copy.	Routine hash verification of trusted inputs; comparing against known-good versions; keeping backups and restoring rather than fixing by hand; enforcing trust boundaries before AI ingestion.